

TALLER 6

SEMINARIO

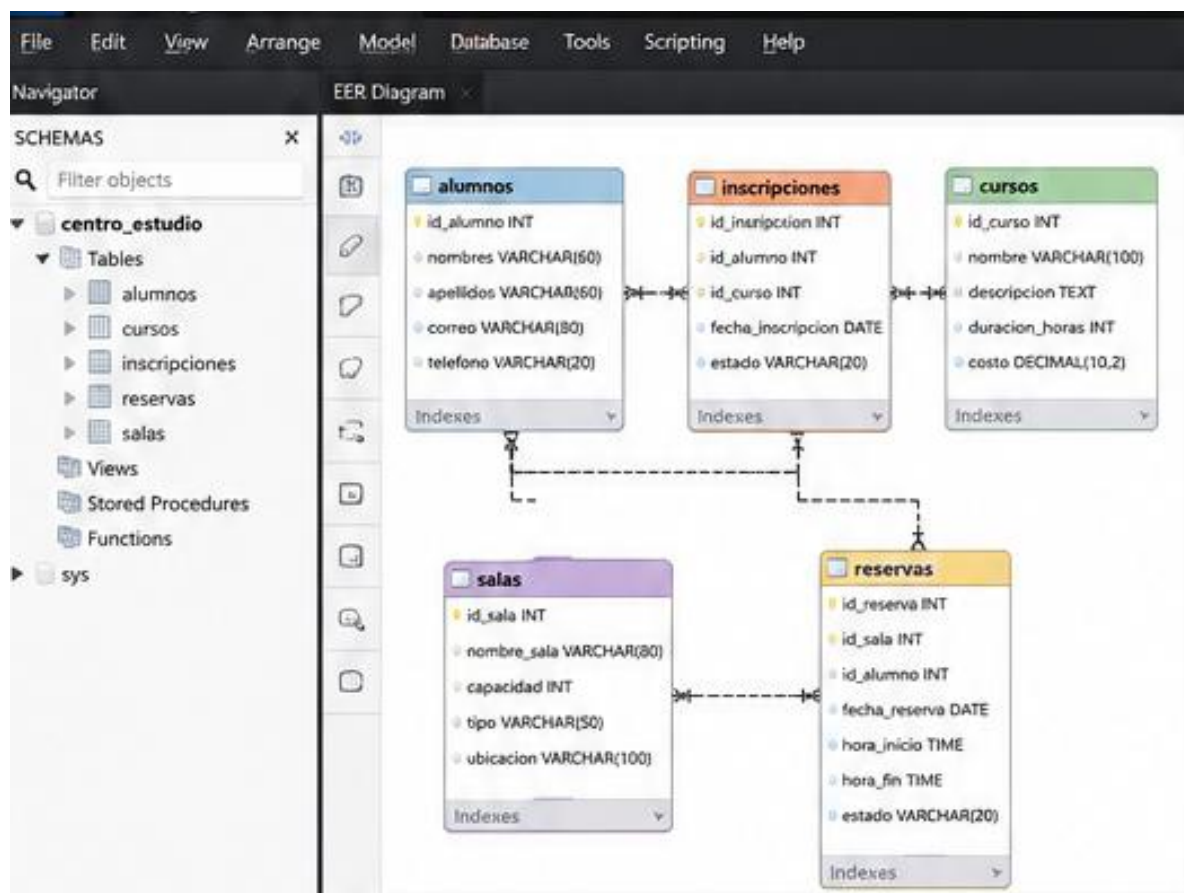
MARIA DEL MAR FIERRO CASTRO

2026

El siguiente taller presenta el diseño e implementación de una base de datos orientada a la administración de un centro de estudio tecnológico. El sistema permite registrar estudiantes, salas de estudio, servicios adicionales y pagos realizados por las reservas.

A diferencia del modelo original de coworking, este proyecto se enfoca en un entorno académico y tecnológico, manteniendo las relaciones necesarias entre entidades, especialmente la relación muchos a muchos entre reservas y servicios.

Diagrama er generado desde DBML que muestra tablas y relaciones



Definición DBML con tablas principales y claves y creación de tablas

```
SQL File 1* x
Limit to 1000 rows

1 -- 1. Tabla alumnos
2 CREATE TABLE alumnos (
3     id_alumno INT PRIMARY KEY,
4     nombres VARCHAR(80) NOT NULL,
5     apellidos VARCHAR(50) NOT NULL,
6     correo VARCHAR(80) UNIQUE,
7     telefono VARCHAR(20)
8 );
9 -- 2. Tabla cursos
10 CREATE TABLE cursos (
11     id_curso INT PRIMARY KEY,
12     nombre VARCHAR(100) NOT NULL,
13     descripcion TEXT,
14     duracion_horas INT,
15     costo DECIMAL(10,2)
16 );
17 -- 3. Tabla inscripciones (relacion muchos a muchos entre alumnos y cursos)
18 CREATE TABLE inscripciones (
19     id_inscripcion INT PRIMARY KEY,
20     id_alumno INT NOT NULL,
21     id_curso INT NOT NULL,
22     fecha_inscripcion DATE NOT NULL,
23     estado VARCHAR(20) DEFAULT 'activa',
24     FOREIGN KEY (id_alumno) REFERENCES alumnos(id_alumno),
25     FOREIGN KEY (id_curso) REFERENCES cursos(id_curso)
26 );
27 -- 4. Tabla salas
28 CREATE TABLE salas (
29     id_sala INT PRIMARY KEY,
30     nombre_sala VARCHAR(80) NOT NULL,
31     capacidad INT,
32     tipo VARCHAR(50),
33     ubicacion VARCHAR(100)
34 );
35 -- 5. Tabla reservas
36 CREATE TABLE reservas (
37     id_reserva INT PRIMARY KEY,
38     id_sala INT NOT NULL,
39     id_alumno INT NOT NULL,
```

Insertión de Datos

```
1 -- Alumnos
2 INSERT INTO alumnos (id_alumno, nombres, apellidos, correo, telefono) VALUES
3 (1, 'Lucia', 'Ramirez', 'lucia.ramirez@mail.com', '3123456788'),
4 (2, 'Juan', 'Perez', 'juan.perez@mail.com', '3112345678'),
5 (3, 'Maria', 'Gomez', 'maria.gomez@mail.com', '3132345678');
6
7 -- Cursos
8 INSERT INTO cursos (id_curso, nombre, descripcion, duracion_horas, costo) VALUES
9 (1, 'Programación Básica', 'Introducción a la lógica de programación', 40, 250000),
10 (2, 'Diseño Web', 'HTML, CSS y fundamentos de JS', 30, 200000),
11 (3, 'Bases de Datos', 'Modelado y SQL', 35, 220000);
12
13 -- Inscripciones
14 INSERT INTO inscripciones (id_inscripcion, id_alumno, id_curso, fecha_inscripcion, estado) VALUES
15 (1, 1, 1, '2024-05-01', 'activa'),
16 (2, 1, 3, '2024-05-03', 'activa'),
17 (3, 2, 2, '2024-05-03', 'activa'),
18 (4, 3, 2, '2024-05-04', 'activa');
19
20 -- Salas
21 INSERT INTO salas (id_sala, nombre_sala, capacidad, tipo, ubicacion) VALUES
22 (1, 'Sala A', 20, 'Informática', 'Piso 1'),
23 (2, 'Sala B', 15, 'Multimedia', 'Piso 1'),
24 (3, 'Sala C', 30, 'Informática', 'Piso 2');
25
26 -- Reservas
27 INSERT INTO reservas (id_reserva, id_sala, id_alumno, fecha_reserva, hora_inicio, hora_fin,
28 estado) VALUES
29 (1, 1, 1, '2024-05-10', '08:00:00', '10:00:00', 'confirmada'),
30 (2, 2, 2, '2024-05-10', '10:30:00', '12:00:00', 'confirmada'),
31 (3, 3, 3, '2024-05-11', '14:00:00', '16:00:00', 'pendiente');
```

Consulta de Relación Muchos a Muchos

1	-- Alumnos inscritos en cada curso
2	SELECT c.nombre AS curso, COUNT(i.id_alumno) AS total_alumnos
3	FROM cursos c
4	JOIN inscripciones i ON c.id_curso = i.id_curso
5	GROUP BY c.id_curso, c.nombre;
1 row(s) returned	
1	-- Salas reservadas en una fecha específica
2	SELECT s.nombre_sala, r.hora_inicio, r.hora_fin, a.nombres
3	FROM reservas r
4	JOIN salas s ON r.id_sala = s.id_sala
5	JOIN alumnos a ON r.id_alumno = a.id_alumno
9	WHERE r.fecha_reserva = '2014-05-10';
1 row(s) returned	
1	-- Cursos en los que está inscrito cada alumno
2	SELECT a.nombres, c.nombre AS curso, i.estado
3	FROM alumnos a
4	JOIN inscripciones i ON a.id_alumno = i.id_alumno
5	JOIN cursos c ON i.id_curso = c.id_curso
9	ORDER BY a.nombres;
1 row(s) returned	

Verificación de Integridad

1	-- A. Intento de borrar un curso con inscripciones (debe fallar)
	DELETE FROM cursos WHERE id_curso = 1;
Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails ('centro_estudio'.inscripciones, CONSTRAINT 'inscripciones_ibfk_2' FOREIGN KEY ('id_curso') REFERENCES 'cursos' ('id_curso')).	
0 row(s) affected	
3	
3	-- B. Intento de reservar una sala inexistente (debe fallar)
4	INSERT INTO reservas (id_reserva, id_sala, id_alumno, fecha_reserva, hora_inicio, hora_fin, estado) VALUES (4, 99, 1, '2024-05-12', '09:00:00', '11:00:00', 'confirmada');
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('centro_estudio'.reservas, CONSTRAINT 'reservas_ibfk_1' FOREIGN KEY ('id_sala') REFERENCES 'salas' ('id_sala'))	
0 row(s) affected	
8	
3	-- C. Requisitos mínimos verificados
5 tablas creadas	
Relación muchos a muchos implementada (alumnos - cursos)	
Integridad referencial funcionando	
Datos insertados correctamente	
Consultas con resultados válidos	

Reflexion

El desarrollo de este taller permitió comprender la importancia del modelado relacional en el diseño de bases de datos. La implementación de llaves primarias, llaves foráneas y relaciones muchos a muchos facilita la organización correcta de la información y evita inconsistencias.

Uno de los aspectos más importantes fue la creación de la tabla intermedia `reserva_beneficio`, ya que permitió asociar múltiples beneficios a una sola reserva. Además, las restricciones referenciales demostraron cómo SQL protege la integridad de los datos evitando eliminaciones incorrectas.

Finalmente, este ejercicio mostró cómo un modelo conceptual puede transformarse en una solución funcional y escalable para sistemas reales relacionados con centros tecnológicos y espacios de estudio.